

Dosya Organizasyonu II

CEN207 Veri Yapıları — Hafta 14

Dr. Öğr. Üyesi Uğur CORUH · 2026-2027 Güz

Bugünün planı (3 saat)

Saat	Konu
1	Dizinler Anim 1–2 · ISAM Anim 3 · B-ağacı ekleme Anim 4
2	B-ağacı arama/silme Anim 5–6 · B+-ağacı Anim 7
3	Genişleyebilir/doğrusal hashleme Anim 8–9 · dış sıralama Anim 10–11

Öğrenme çıktıları: ÖÇ.1 (temel veri yapılarını açıklama) · ÖÇ.2 (karmaşıklık analizi) · ÖÇ.6 (dosya tabanlı depolama tasarımı) · ÖÇ.7 (doğru yapıyı seçme)

Bu haftanın kavramları — nerede

Kavram	Nerede
Birincil/ikincil dizin, ISAM	Bölüm 1–3
B-ağacı ekleme/arama/silme, B+-ağacı	Bölüm 4–7
Genişleyebilir hashleme, doğrusal hashleme	Bölüm 8–9
Dış birleştirmeli sıralama, yerine koyarak seçim	Bölüm 10–11

Kod örnekleri nasıl çalışır

- Her fikrin tam bir **C** ve **Java** programı var
- C: `gcc -std=c11 -Wall -Wextra -o /tmp/x file.c && /tmp/x`
- Java: `javac -d /tmp/j File.java && java -cp /tmp/j File`
- Kaynaklar: `code/week-14/c/` ve `code/week-14/java/`
- İki program GERÇEK dosyalar yaratır, yalnız kendi kurduğu bir lab klasöründe

Özet — Hafta 13: sayfalar, sıralı dosyalar, kova hashleme

- Bir **sayfa**, disk G/Ç'sinin birimidir — bu haftanın da para birimi
- **Sıralı bir dosya**: sayfalar üzerinde ikili arama, $O(\log(n/B))$
- **Kova-hashli bir dosya**: $\sim O(1)$ arama, ama taşma zincirleri
- Bugün: büyümeyi VE aramayı birlikte ucuzlatan yapılar

Özet — Hafta 4: ağaçlar ve yeniden dengeleme

- Bir İAA düşmanca bir ekleme sırasında $O(n)$ 'e bozulabilir
- Yeniden dengeleme: bir değişiklikten sonra yerel olarak yeniden yapılandır
- Bir B-ağacı koca **sayfaları** dengeler, tek anahtarlı düğümleri değil
- Sayfaları bölmek ve birleştirmek yüksekliği hep $O(\log n)$ tutar

Haftanın haritası — genel bakış

Dizinler	Dengeli ağaçlar	Dinamik hashleme	Dış sıralama
Birincil, ikincil, ISAM	B-ağacı, B+-ağacı	Genişleyebilir, doğrusal	Birleştirmeli sıralama, yerine koyarak seçim

1. Birincil (Seyrek) Dizininler

Başlangıç sorusu

Bir dosyanın sayfaları üzerinde ikili arama zaten $O(\log(n/B))$ okumaya mal oluyor. RAM'e sığacak kadar küçük bir yapı bunu daha da kısaltabilir mi?

Sezgi — bir kitabın bölüm sekmeleri

- Bir kelimeyi bulmak için her sayfayı çevirmek: çok yavaş
- Her bölümün ilk kelimesini tutan bir **sekm**e
- Sekmeleri kontrol et (bedava, elinizde), TEK bölümü aç
- Bir seyrek dizinin her disk sayfası için yaptığı tam olarak bu

Fikir: SAYFA başına tek girdi

- Dizin: `(ilk_anahtar, sayfa)` çiftleri, veri sayfası başına bir
- Dosyanın kendisiyle aynı şekilde sıralı
- Küçük: verinin `1/B` 'i kadar — RAM'e sığar, ücretsiz taranır
- `find_page` : `first_key <= key` olan son girdi

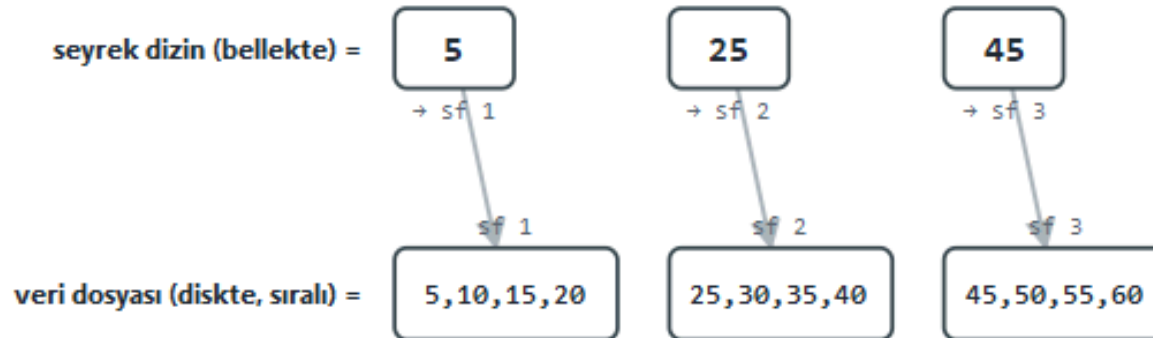
search_key: dizin taraması, sonra TEK sayfa okuması

- `find_page`, anahtar her `first_key` 'den küçükse `-1` döndürür
- `-1` : **hiç disk erişimi gerekmez**
- Aksi halde: o tek sayfayı oku, anahtarı içinde tara
- Toplam gerçek disk maliyeti: en çok **bir** sayfa okuması

Birincil dizin, adım adım

BLOCK = 4

okuma: 2 yazma: 3

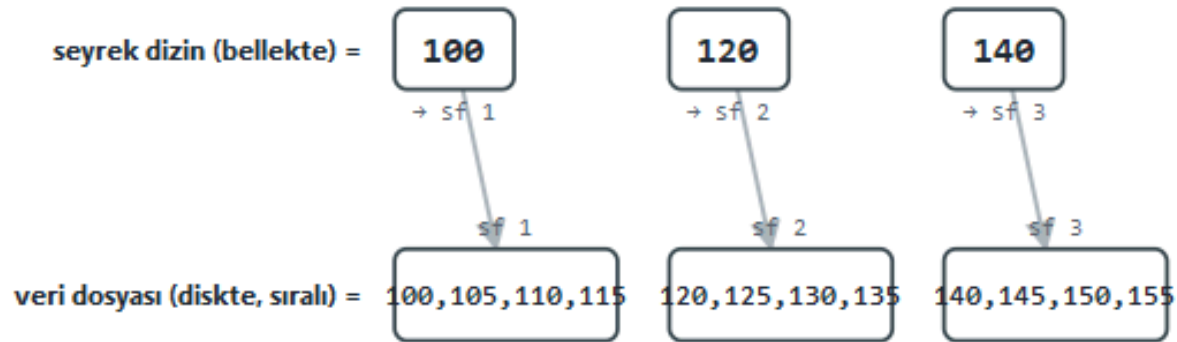


Bitti: 3 arama, toplam 2 disk okuması (yazma: 3). Her arama en çok 1 sayfa okur: seyrek dizin sayesinde $O(n/B)$ dizin taraması (bellekte, ücretsiz) + $O(B)$ sayfa içi tarama -- toplam disk maliyeti yalnız $O(1)$ sayfa.

Uç durum — her sorgu aralığının altında

BLOCK = 4

okuma: 0 yazma: 3



Bitti: 3 arama, toplam 0 disk okuması (yazma: 3). Her arama en çok 1 sayfa okur: seyrek dizin sayesinde $O(n/B)$ dizin taraması (bellekte, ücretsiz) + $O(B)$ sayfa içi tarama -- toplam disk maliyeti yalnız $O(1)$ sayfa.

Kod — find_page

```
int find_page(IndexEntry index[], int idx_n, int key) {  
    int page = -1;  
    for (int i = 0; i < idx_n; i++) {  
        if (index[i].first_key <= key)  
            page = index[i].page;  
        else  
            break;  
    }  
    return page;  
}
```

Kod — search_key

```
bool search_key(int data[][MAX_BLOCK], const int page_len[],
                IndexEntry index[], int idx_n, int key, int *out_page) {
    int page = find_page(index, idx_n, key);
    if (page == -1) return false;
    for (int i = 0; i < page_len[page]; i++)
        if (data[page][i] == key) { *out_page = page; return true; }
    *out_page = page;
    return false;
}
```

Karmaşıklık

- Dizin taraması: en kötü durumda $O(n/B)$ karşılaştırma, ama **sıfır** disk G/Ç'si
- Garanti edilen tek disk okuması: tek veri sayfası
- Arama başına gerçek toplam G/Ç: **$O(1)$** sayfa okuması
- Dizin yeri: $O(n/B)$ — kayıtla değil, sayfayla orantılı

Sık yapılan hatalar

- Sıralı olmayan bir dosyada seyrek dizinin işe yarayacağını varsaymak
- "-1: hiçbir sayfa uymuyor"u "sayfa tarandı, eşleşme yok" ile karıştırmak
- Dizinin kendisinin RAM'e sığmayacak kadar büyümesine izin vermek

Mini soru

Bir seyrek dizin neden kayıt başına değil, yalnız SAYFA başına TEK girdiye ihtiyaç duyar?

Cevap

Bir sayfa okuması + sayfa-içi tarama, o sayfadaki her kaydı zaten bulur. Sayfa başına daha fazla girdi hiçbir ek okuma kazandırmazdı.

2. İkincil (Yoğun) Dizinler

Başlangıç sorusu

Birincil dizin dosyanın kendi sıralama anahtarında çalışır. Bir dizin FARKLI, tekrar eden bir öznitelikte de yardımcı olabilir mi?

Sezgi — bir kütüphanenin konu kataloğu

- Kitaplar yer numarasına göre rafta durur (birincil sıra)
- Konu göre sıralı bir katalog kartı **kitap başına** vardır
- Aynı konudaki kartlar birbirinin yanına düşer
- Her kart yine de kitabın gerçek raf konumuna işaret eder

Fikir: anahtara göre sıralı, KAYIT başına bir girdi

- (anahtar, konum) çiftleri, kayıt başına BİR — sayfa başına değil
- İkincil anahtarın kendisine göre sıralı
- Dizin sıralı olduğundan tekrarlar **bitişik** sona erer
- Bir eşleşme kümesi: anahtar değişene kadar ileri tara

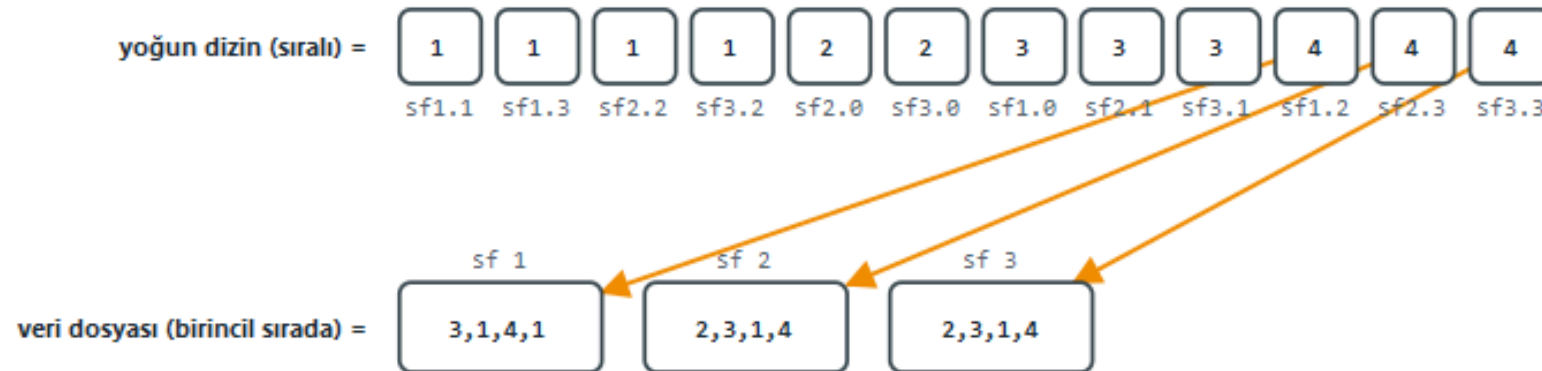
search_dense: tüm bir kümeyi topla

- Bir eşleşme başlamadan önceki girdiler atlanır
- Bir eşleşmenin içindeyken her ardışık girdi toplanır
- Bir eşleşmeden sonra FARKLI bir anahtar: küme bitti
- Dokunulan her farklı sayfa için en çok bir yeni okuma

İkincil dizin, adım adım

BLOCK = 4

okuma: 6 yazma: 3

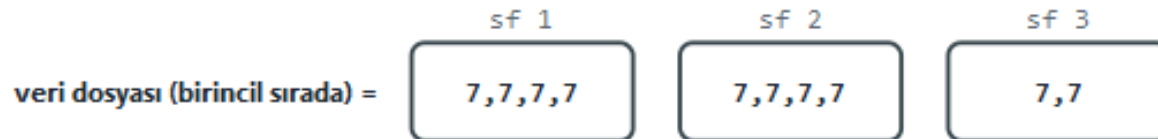


Bitti: 3 arama, toplam 6 disk okuması. Yoğun dizin sıralı olduğu için tekrarlar bitişik durur: $O(n)$ dizin taraması (bellekte) + eşleşme başına en çok 1 yeni sayfa okuması.

Uç durum — tüm kayıtlar bir anahtarı paylaşıyor

BLOCK = 4

okuma: 3 yazma: 3



Bitti: 2 arama, toplam 3 disk okuması. Yoğun dizin sıralı olduğu için tekrarlar bitişik durur: $O(n)$ dizin taraması (bellekte) + eşleşme başına en çok 1 yeni sayfa okuması.

Kod — search_dense

```
int search_dense(const IndexEntry index[], int n, int key,
                 int matches[], int max_matches) {
    int count = 0;
    for (int i = 0; i < n; i++) {
        if (index[i].key == key) {
            if (count < max_matches) matches[count] = index[i].slot;
            count++;
        } else if (count > 0) {
            break;
        }
    }
    return count;
}
```

Karmaşıklık

- Yazıldığı gibi doğrusal tarama: en kötü durumda $O(n)$
- Üretim sürümü: ilk eşleşmeye ikili arama, $O(\log n + m)$
- Disk maliyeti: eşleşen her farklı sayfa için en çok bir okuma
- Yer: $O(n)$ — sayfa başına değil, kayıt başına bir girdi

Sık yapılan hatalar

- Yoğun dizinin VERİ dosyasının sırasına göre sıralı olduğunu varsaymak
- Bir küme başlamadan önce ilk eşleşmeyende taramayı kırmak
- Yoğun bir dizinin $O(n/B)$ değil $O(n)$ yer kapladığını unutmak

Mini soru

Yoğun bir dizin neden seyrek dizinin
sayfa başına birine karşı, KAYIT başına bir girdiye ihtiyaç duyar?

Cevap

Veri dosyası ikincil anahtara göre sıralı **DEĞİLDİR**. Yalnızca kayıt başına bir dizin girdisi doğru bir eşleşme listesi garanti edebilir.

3. ISAM: Çok Seviyeli Dizin + Taşma

Başlangıç sorusu

Bölüm 1–2, dosyanın hiç değişmeyeceğini varsaydı. Gerçek dosyalar büyür. Sayfası zaten dolu bir kayıt nereye gider?

Kısa bir tarihçe — IBM, 1960'lar

- **ISAM** (Dizinli Sıralı Erişim Yöntemi)
- IBM'in erken mainframe veritabanları için üretim cevabı
- Onlarca yıl ticari veri işlemede kullanıldı
- Seyrek bir dizini bir taşıma alanıyla birleştirir

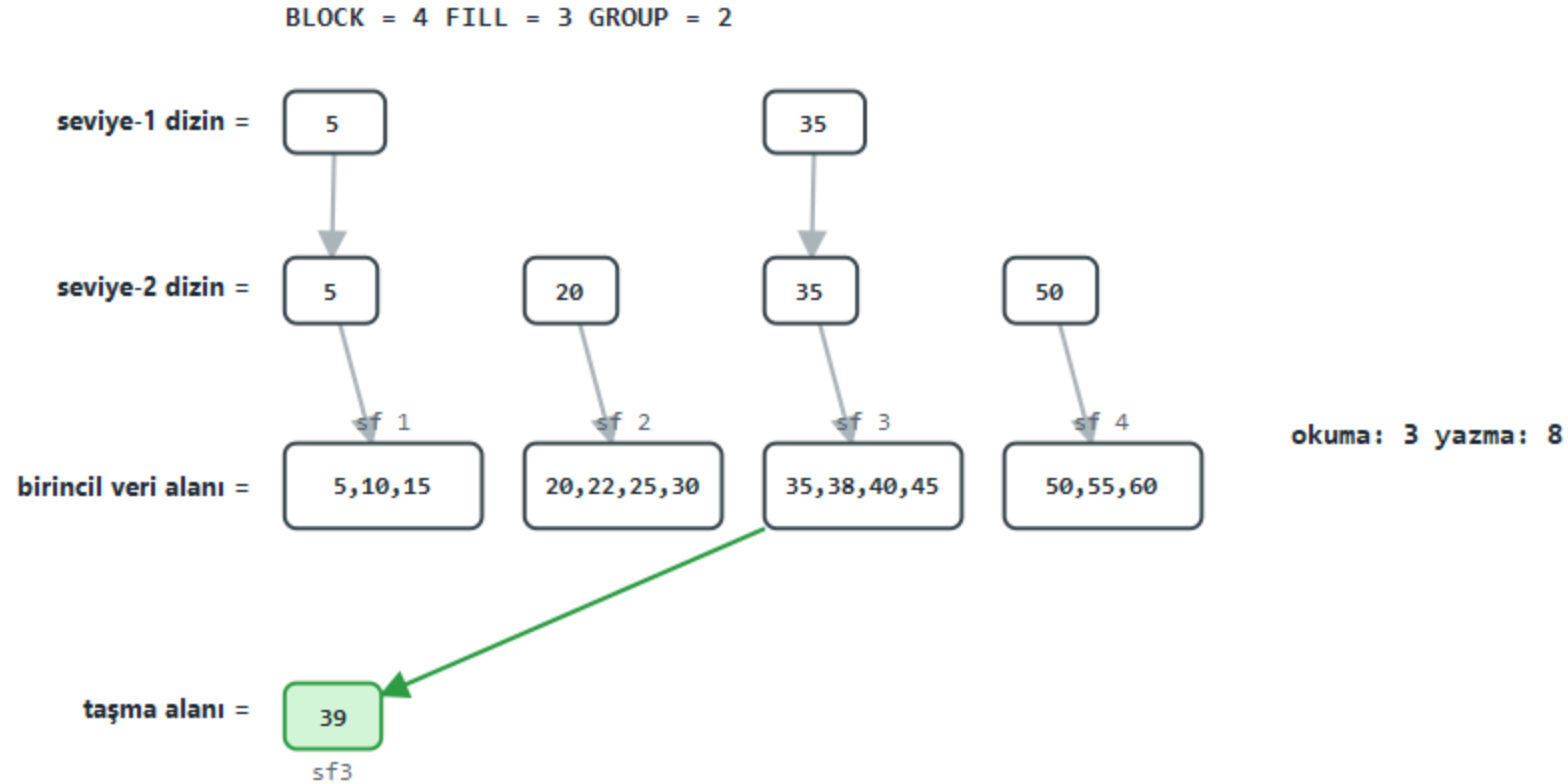
Fikir: dizinin dizinini tut, bir kaçış valfi ekle

- Seviye-2 dizin: sayfa başına bir girdi (Bölüm 1 gibi)
- Seviye-1 dizin: her GROUP seviye-2 girdisini gruplar
- İki kısa tarama, tek uzun bir doğrusal taramanın yerini alır
- Dolu bir sayfanın yeni anahtarı bağlı bir TAŞMA zincirine gider

Taşma zincirleri: şimdi ucuz, sonra pahalı

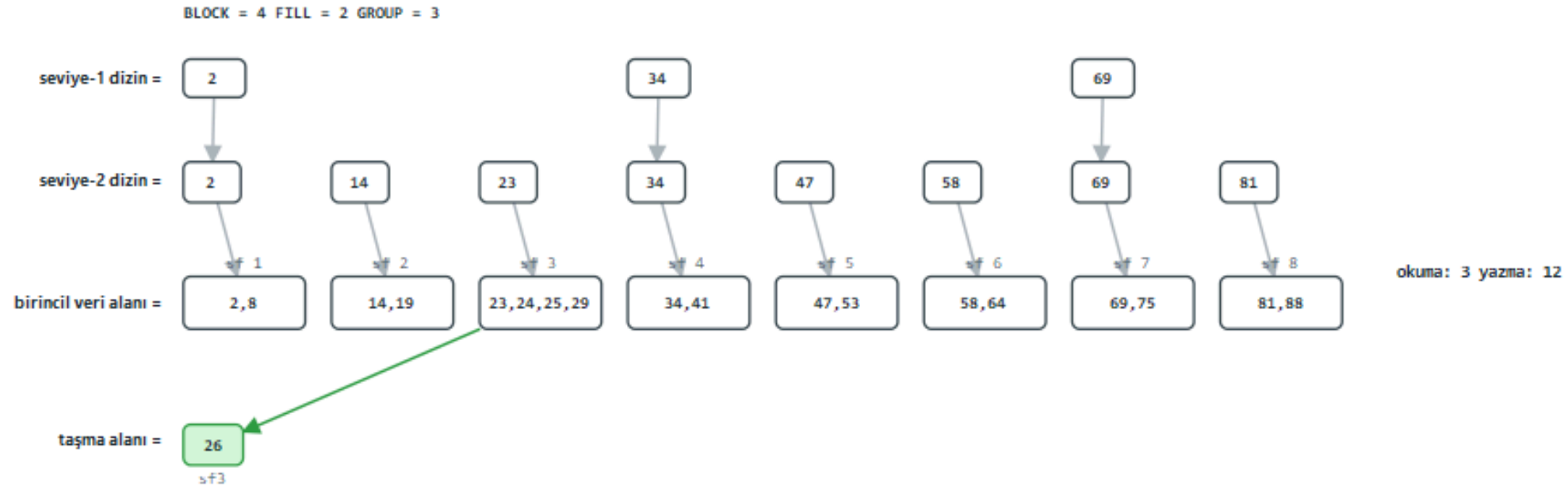
- Taşma zincirine ekleme: sona ekle, yerel olarak $O(1)$ amortize
- Arama, ev sayfasında değilse TÜM zinciri yürümeli
- Uzun zincirler: arama başarımı zamanla bozulur
- Çözüm: periyodik **yeniden düzenleme** dosyayı sıfırdan kurar

ISAM, adım adım



Bitti: 3 ekleme, toplam 3 okuma / 8 yazma. Taşma zincirleri uzadıkça arama yavaşlar -- bu yüzden gerçek ISAM dosyaları düzenli olarak yeniden düzenlenir (reorganize).

Uç durum — zincirlenen taşma, aynı sayfa



Bitti: 3 ekleme, toplam 3 okuma / 12 yazma. Taşma zincirleri uzadıkça arama yavaşlar -- bu yüzden gerçek ISAM dosyaları düzenli olarak yeniden düzenlenir (reorganize).

Kod — isam_insert

```
void isam_insert(int key) {
    int g = find_group(l1_key, l1_n, key);
    int page = find_page(l2_key, g * GROUP, group_hi(g), key);
    read_page(page);
    if (page_len[page] < BLOCK) {
        insert_sorted(page, key);
        write_page(page);
    } else {
        int walked = walk_overflow_chain(page);
        append_overflow(page, key);
    }
}
```

Karmaşıklık

- Arama: $O(L)$ bellek-içi karşılaştırma + 1 ev-sayfası okuması
- Bulunamazsa, yürünen her taşma düğümü için bir okuma daha
- Ekleme: aynı arama + 1 yazma (doğrudan) ya da 2 yazma (taşma)
- Zincirler uzadıkça arama bozulur — yeniden düzenlemenin gerekçesi

Sık yapılan hatalar

- Taşma zincirlerinin sınırsız büyümesine izin verip hiç yeniden düzenlememek
- Yeni bir taşma düğümünü yanlış sayfanın zincirine eklemek
- Doğru GRUBU bulup, sayfalarını taramadan durmak

Mini soru

ISAM neden ÇOK SEVİYELİ bir dizine ihtiyaç duyar, Bölüm 1'in dizini yalnız birini kullanırken?

Cevap

"Küçük" tek seviyeli bir dizin bile koca bir dosya için çok büyüyebilir. Seviye-1 altında gruplamak en üst taramayı küçük tutar.

4. B-Ağaçları: Ekleme

Başlangıç sorusu

ISAM'ın dizin seviyeleri kurulum zamanında sabittir. Ya dizinin kendisi hiç yeniden düzenleme olmadan büyüyüp dengelenebilseydi?

Kısa bir tarihçe — Bayer ve McCreight, 1972

- Rudolf Bayer, Edward McCreight, Boeing Araştırma Lab.
- "Organization and Maintenance of Large Ordered Indexes"
- "B" Bayer, Boeing, ya da "balanced" olarak okunur — hiç kesinleşmedi
- ISAM'ın sabit yapısının çözemediğini çözdü: kanıtlanabilir denge

Sezgi — tepeden büyüyen bir dosya dolabı

- Her çekmece (sayfa) sıralı birkaç dosya (anahtar) tutar
- Dolu bir çekmece iki yarı dolu çekmeceye bölünür
- Ortadaki dosya YUKARIDAKİ çekmeceye çıkar
- Dolap yalnız zorunda kalınca YENİ BİR ÜST çekmece kazanır

Derece m : sayfa başına kaç anahtar

- Derece m : en çok $m-1$ anahtar, m çocuk / düğüm
- Kök daha az tutabilir; diğer her düğüm en az $\lceil m/2 \rceil - 1$
- Ekleme, tam bir aramanın yapacağı gibi bir yaprağa iner
- Yaprak artık m anahtar tutuyorsa: **böl**

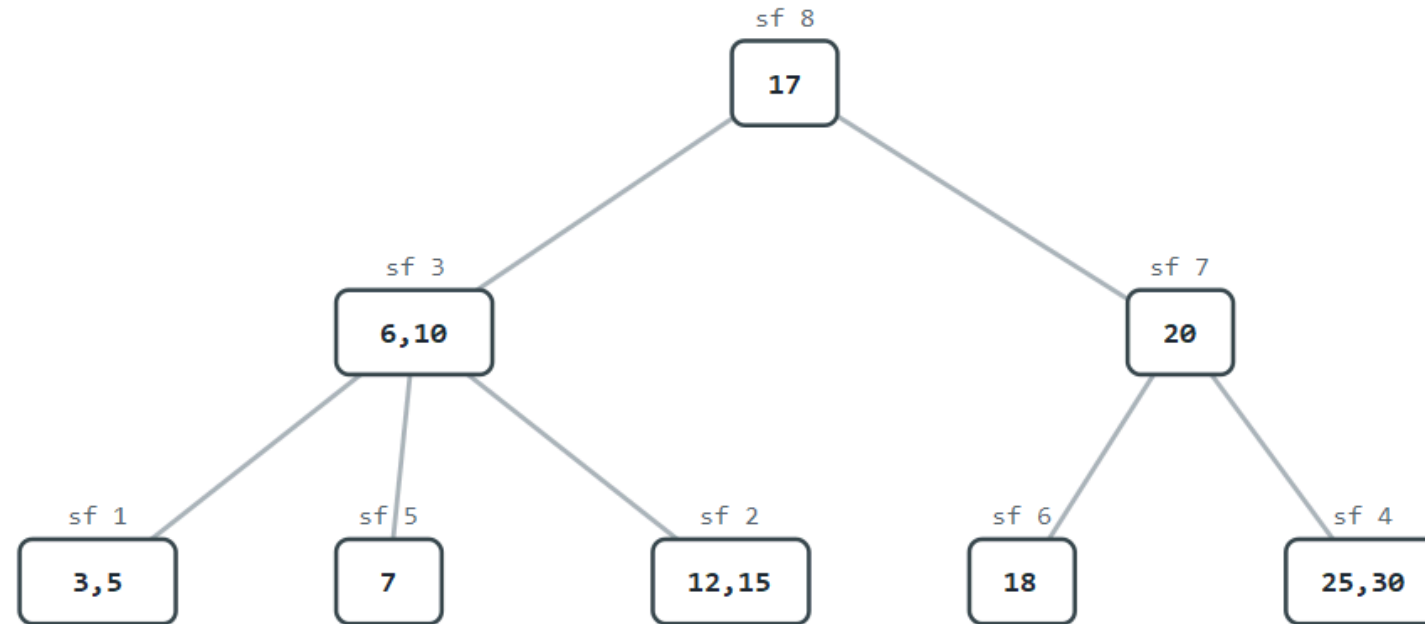
split: ortancayı yukarı it

- `mid = n/2` ; ortanca anahtar ebeveyne itilir
- Sol yarı `keys[0..mid-1]` 'i tutar, sağ yarı kalanı alır
- EBEVEYN de taşıyorsa: onu da böl
- KÖK bölünürse: yeni bir kök ağacı bir seviye büyütür

B-ağacı ekleme, adım adım

ORDER = 4 (max 3 keys/node)

okuma: 20 yazma: 28



Bitti: 12 ekleme, sonuçta 8 sayfa, yükseklik 2, 20 okuma / 28 yazma. B-ağacı hep DENGELİ kalır: her yaprak aynı derinlikte.

Uç durum — order=12, hiç bölünmüyor

ORDER = 12 (max 11 keys/node)

okuma: 10 yazma: 11

sf 1

3,5,7,12,16,23,29,38,41,44

Bitti: 10 ekleme, sonuçta 1 sayfa, yükseklik 0, 10 okuma / 11 yazma. B-ağacı hep DENGELİ kalır: her yaprak aynı derinlikte.

Kod — insert_sorted ve split

```
void insert_sorted(Node *node, int key) {  
    int i = node->n - 1;  
    while (i >= 0 && node->keys[i] > key) {  
        node->keys[i + 1] = node->keys[i]; i--;  
    }  
    node->keys[i + 1] = key;  
    node->n++;  
}
```

Kod — b_tree_insert

```
void b_tree_insert(BTree *t, int key) {
    Node *leaf = find_leaf(t->root, key);
    insert_sorted(leaf, key);
    Node *cur = leaf;
    while (cur->n == ORDER) {
        int median; Node *right = split(cur, &median);
        if (cur->parent == NULL) { t->root = new_root(median, cur, right); return; }
        insert_sorted(cur->parent, median);
        attach_child(cur->parent, right);
        cur = cur->parent;
    }
}
```

Karmaşıklık

- Yükseklik: $O(\log_m n)$ — derece 100, 1 milyar kayıt: yalnız 5 seviye
- Ekleme: yaprağı bulmak için $O(\log_m n)$ okuma
- Artı en çok $O(\log_m n)$ bölünme, her biri $O(m)$ iş
- En kötü durum (köke kaskad): $O(m \log_m n)$

Sık yapılan hatalar

- Yalnız yaprağı bölüp, ebeveynin de taşabileceğini unutmak
- Ağacı yanlış uçtan büyütmek (tepe yerine alttan)
- Sıradan verinin bile sürekli bölünmesine yol açacak kadar küçük bir derece seçmek

Mini soru

Bir B-ağacının yüksekliği neden düşmanca bir ekleme sırasında bile $O(\log_m n)$ kalır?

Cevap

Her bölünme yereldir; ağaç yalnızca bir bölünme **KÖKE** ulaştığında büyür. Hiçbir ekleme sırası uzun bir zincir yaratamaz.

5. B-Ağacı: Arama

Başlangıç sorusu

Ekleme dengeli bir ağaç kurar. Zaten var olan birinde, bir anahtarı bulmak — ya da yokluğunu kanıtlamak — kaç sayfaya mal olur?

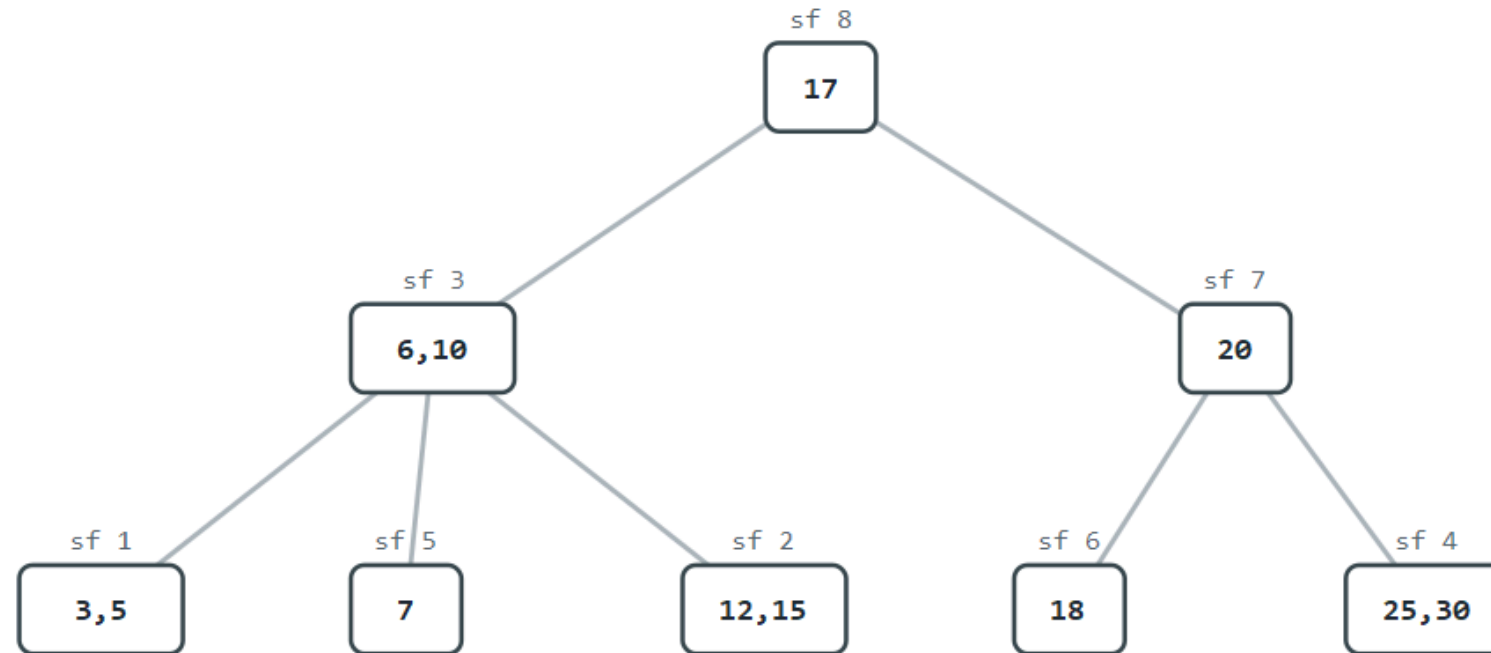
Fikir: her sayfada karşılaştırarak in

- Kökte başla; (küçük) sıralı anahtar listesini tara
- Tam eşleşme: bitti
- Eşleşme yok, ve bu bir YAPRAK: anahtar olamaz — yok
- Eşleşme yok, yaprak değil: tam olarak bir çocuğa in

B-ağacı arama, adım adım

ORDER = 4

okuma: 7



Bitti: 3 arama, toplam 7 disk okuması. Her arama en çok $\text{height} + 1 = 3$ sayfa okur: $O(\log_{\text{ORDER}} n)$.

Uç durum — tek düğüm, her arama 1 okuma

ORDER = 12

okuma: 3

sf 1

3, 5, 7, 12, 16, 23, 29, 38, 41, 44

Bitti: 3 arama, toplam 3 disk okuması. Her arama en çok $\text{height} + 1 = 1$ sayfa okur: $O(\log_{\text{ORDER}} n)$.

Kod — b_tree_search

```
bool b_tree_search(Node *node, int key, Node **out_node) {  
    if (node == NULL) return false;  
    int i = 0;  
    while (i < node->n && key > node->keys[i]) i++;  
    if (i < node->n && key == node->keys[i]) {  
        *out_node = node; return true;  
    }  
    if (node->leaf) return false;  
    return b_tree_search(node->child[i], key, out_node);  
}
```

Karmaşıklık

- En kötü durumda $O(\log_m n)$ sayfa okuması — seviye başına bir
- Bir sayfa içinde: ikili aranırsa $O(\log m)$, doğrusal $O(m)$
- Anahtar bulunsun ya da yokluğu kanıtlandsın, aynı maliyet
- Bu, Bölüm 4'ün dengeli eklemesinin doğrudan getirisi

Sık yapılan hatalar

- Bir yaprağın (iklendirilmemiş) çocuk dizisine inmek
- İnilecek çocuk indeksini seçerken birer birlik hata
- "Bulunamadı"nın "bulundu"dan daha az okumaya mal olduğunu varsaymak

Mini soru

B-ağacı araması neden en çok
(yükseklik + 1) okumaya mal olur, hangi anahtar aransa da?

Cevap

Her yaprak tam olarak aynı derinliktedir.

Bir arama ya erken bulur, ya da bir yaprağa kadar iner — asla daha derine değil.

6. B-Ağacında Silme: Ödünç Alma ve Birleştirme

Başlangıç sorusu

Ekleme dolu bir sayfayı böler. Silmenin ayna görüntüsü — iki çok boş sayfayı birleştirmek — bazen önlenebilir mi?

Fikir: sil, sonra eksikliği düzelt

- Yaprak anahtarı: kaydır-sil, basit
- İç anahtar: sıra-içi ÖNCÜL'üyle değiştir
- Öncülün kendi yaprağı, gerçek silmenin olduğu yerdir
- O yaprak eksilirse (çok az anahtar): yukarı doğru düzelt

Eksikliği düzeltmek: önce ödünç al, yalnız gerekirse birleş

- Sol kardeşin yedek anahtarı var mı? **Ödünç al**: ebeveynden döndür
- Yoksa? SAĞ kardeşi aynı şekilde dene
- Hiçbiri yedek vermiyorsa? Bir kardeşle **Birleş**
- Bir birleşme EBEVEYNİ eksiltebilir: kontrol yukarı tekrarlanır

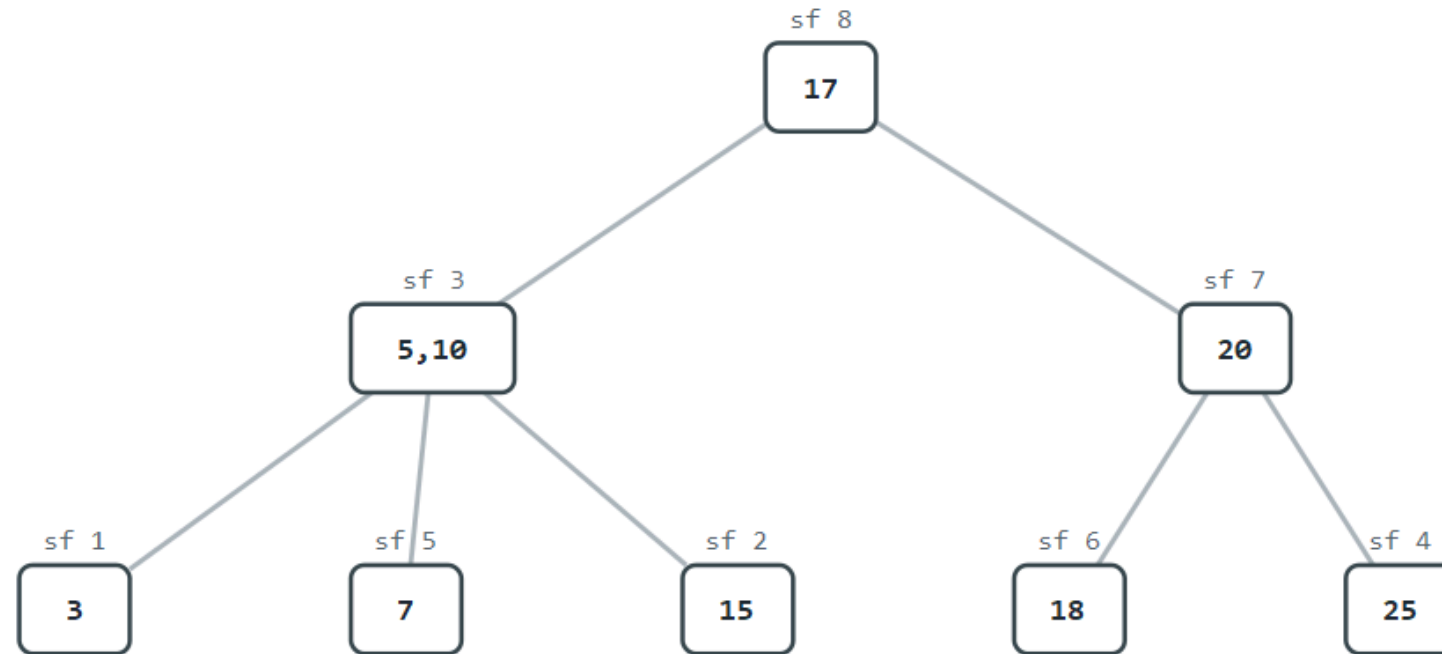
Bir birleşme köke ulaştığında

- Tepedeki bir birleşme kökü tüm anahtarlarından boşaltabilir
- Kökün kalan tek çocuğu yeni kök olur
- Ağaç tam olarak bir seviye **küçülür**
- Bölüm 4'ün kök-bölünme büyümesinin tam ayna görüntüsü

B-ağacı silme, adım adım

ORDER = 4 MIN_KEYS = 1

okuma: 9 yazma: 4

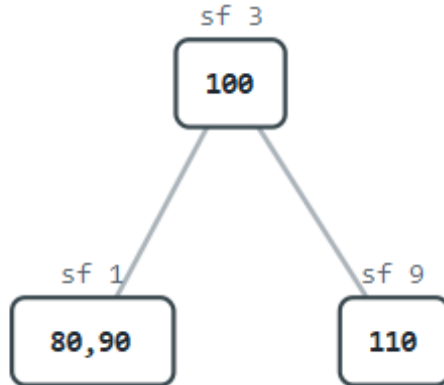


Bitti: 3 silme (0 ödünç, 0 birleştirme), 9 okuma / 4 yazma. Ağaç hâlâ DENGELİ: her yaprak aynı derinlikte.

Uç durum — kök küçülene kadar sil

ORDER = 3 MIN_KEYS = 1

okuma: 19 yazma: 19



Bitti: 7 silme (0 ödünç, 6 birleştirme), 19 okuma / 19 yazma. Ağaç hâlâ DENGELİ: her yaprak aynı derinlikte.

Kod — fix_underflow (ödünç al ya da birleş)

```
void fix_underflow(Node *node) {
    while (node->parent != NULL && node->n < MIN_KEYS) {
        Node *parent = node->parent;
        int idx = child_index(parent, node);
        Node *left = idx > 0 ? parent->child[idx - 1] : NULL;
        Node *right = idx < parent->n ? parent->child[idx + 1] : NULL;
        if (left && left->n > MIN_KEYS) { borrow_from_left(node, parent, left, idx); return; }
        if (right && right->n > MIN_KEYS) { borrow_from_right(node, parent, right, idx); return; }
        if (left) { merge(left, parent, node, idx - 1); node = parent; }
        else { merge(node, parent, right, idx); node = parent; }
    }
}
```

Karmaşıklık

- Anahtarı bulmak: $O(\log_m n)$, aramayla aynı
- Düzeltme: en çok $O(\log_m n)$ birleştirme, seviye başına bir
- Her birleştirme/ödünç: anahtar ve çocukları kaydırmak için $O(m)$
- Ödünç: 3 sayfaya dokunur; birleştirme: bir sayfa + bir anahtar kaldırır

Sık yapılan hatalar

- Tam MIN_KEYS tutan bir kardeşten ödünç almak (yine de eksik kalır)
- Ödünç alınan bir anahtarla birlikte bir çocuk işaretçisini taşımayı unutmak
- Bir iç-düğüm değiştirmesinden sonra yanlış kopyayı silmek

Mini soru

Bir B-ağacı neden ÖDÜNÇ ALMAYI
BİRLEŞTİRMEYE tercih eder, bir kardeşin yedeği olduğunda?

Cevap

Ödünç alma 3 sayfaya dokunur ve hemen **çözülür**. Birleştirme ebeveynden bir sayfa kaldırır, ağaçta daha yukarı yayılabilir.

7. B+-Ağaçları: Yaprak Zincirleri ve Aralık Sorguları

Başlangıç sorusu

"Son 30 gündeki her sipariş" bir ARALIK sorgusudur. Sıradan bir B-ağacı sürekli köke geri tırmanır. Bu önlenabilir mi?

Fikir: anahtarlar yalnız yapraklarda, yapraklar zincir kurar

- İç düğümler yalnız YÖNLENDİRME kopyaları tutar — asla gerçek veri
- Her gerçek anahtar bir YAPRAKTA yaşar
- Her yaprak sağındaki yaprağa bir `next` işaretçisi tutar
- Bir aralık sorgusu: TEK bir kez in, sonra zinciri takip et

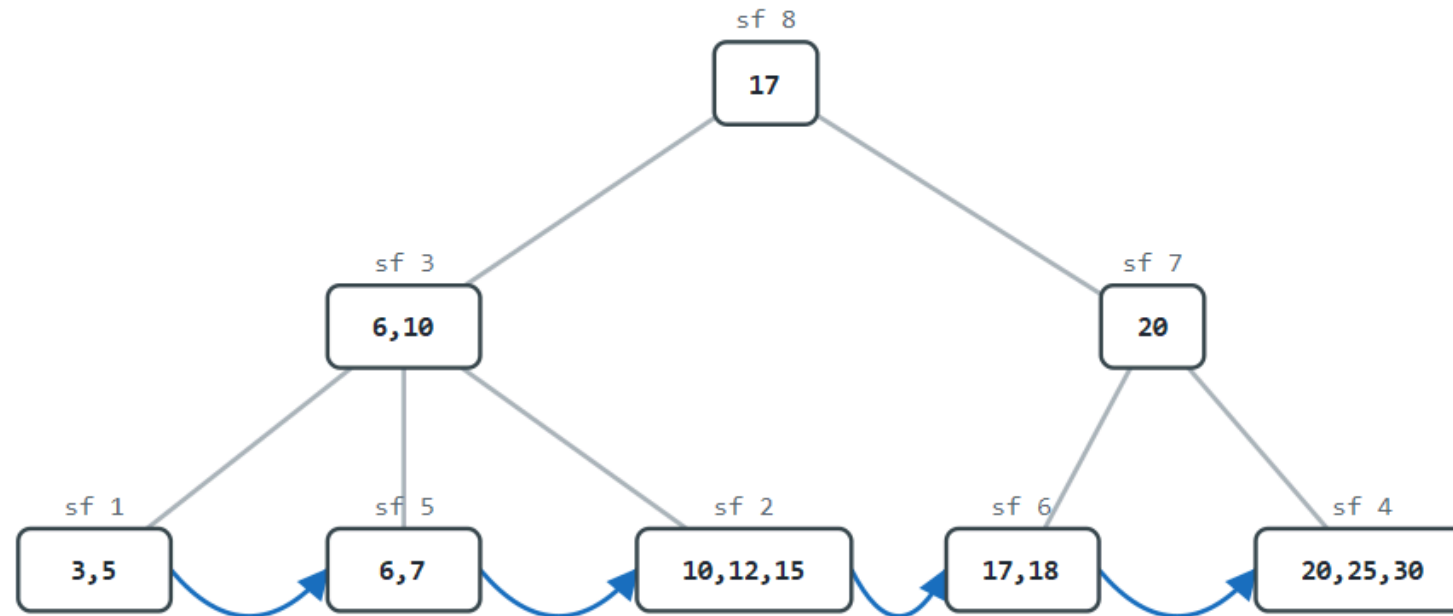
Yaprak bölünmesi kopyalar; iç bölünme kaldırır

- Yaprak bölünmesi: ortanca KOPYALANIR yukarı (sağ yaprakta da kalır)
- İç bölünme: ortanca KALDIRILIR (saf yönlendirme, veri kaybı yok)
- Bölüm 4'ten TEK bu fark, tüm B+-ağacı fikridir
- `next` i doğru eklemek tek yeni muhasebe adımıdır

B+-ağacı, adım adım

ORDER = 4

okuma: 13

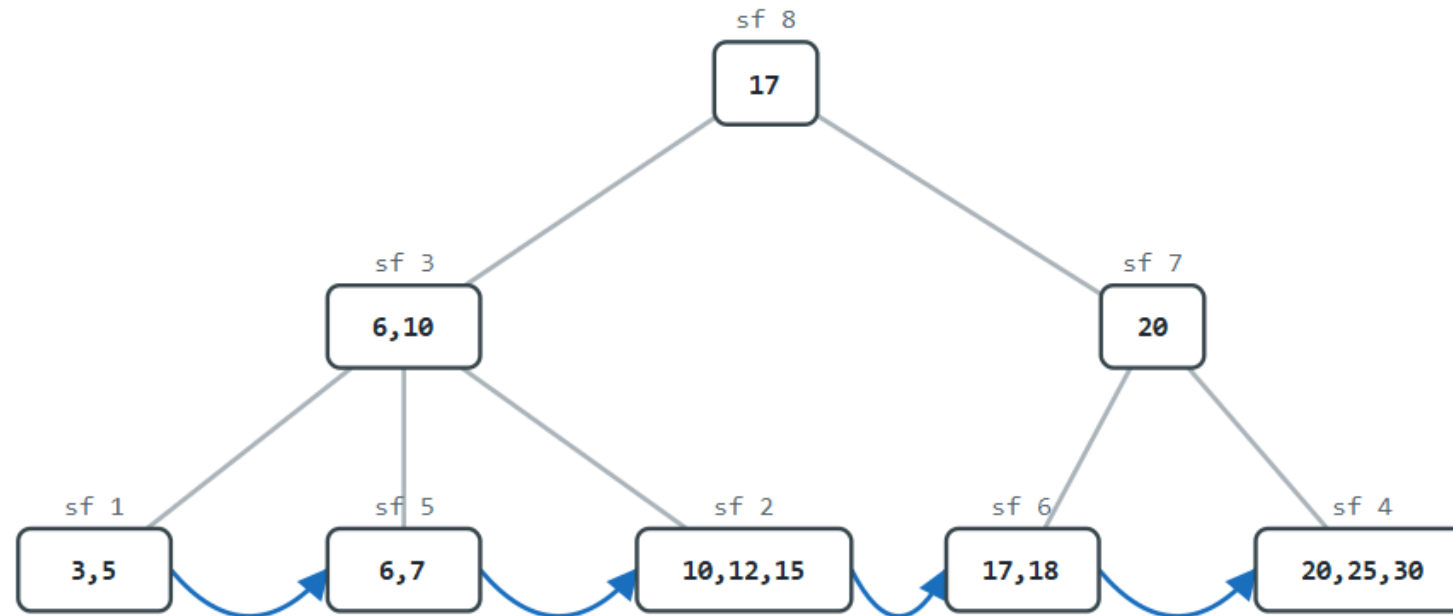


Bitti: 3 aralık sorgusu, toplam 13 okuma. B+-ağacının kazancı: bir kez in, sonra sadece zinciri yürü -- her yaprak için köke dönmek yok.

Uç durum — tüm anahtarları kapsayan bir aralık

ORDER = 4

okuma: 7



Bitti: 1 aralık sorgusu, toplam 7 okuma. B+-ağacının kazancı: bir kez in, sonra sadece zinciri yürü -- her yaprak için köke dönmek yok.

Kod — range_query

```
void range_query(Node *root, int lo, int hi, int out[], int *count) {  
    Node *leaf = find_leaf(root, lo);  
    *count = 0;  
    while (leaf != NULL) {  
        for (int i = 0; i < leaf->n; i++)  
            if (leaf->keys[i] >= lo && leaf->keys[i] <= hi)  
                out[(*count)++] = leaf->keys[i];  
        if (leaf->n > 0 && leaf->keys[leaf->n - 1] > hi) break;  
        leaf = leaf->next;  
    }  
}
```

Karmaşıklık

- Tek arama: $O(\log_m n)$, sıradan bir B-ağacıyla aynı
- k eşleşmeli aralık: $O(\log_m n) + O(k/B)$ zincir okuması
- Sıradan bir B-ağacı: $O(k)$ ayrı kök inişine mal olabilirdi
- Zincir, bu hız kazancının tüm kaynağıdır

Sık yapılan hatalar

- Bir iç bölünmede ortancayı kopyalamak (yalnız yapraklar kopyalar)
- `leaf->next` in üzerine yazmadan önce `next` i eklemeyi unutmak
- Bir aralık sorgusunu tekrarlanan tek-anahtar aramaları olarak çalıştırmak

Mini soru

B+-ağacının aralık sorgusu neden köke tırmanmaktan kaçınabilir, sıradan bir B-ağacınıninki kaçınamazken?

Cevap

Her yaprak kendi bir sonraki yaprağını
doğrudan, $O(1)$ bilir. Sıradan bir
B-ağacında komşu yapraklar arasında böyle bir kısayol yok.

8. Genişleyebilir Hashleme

Başlangıç sorusu

Hafta 13'ün kova-hashli dosyası kova sayısını kurulurken dondurdu. Hashlenmiş bir dosya talep üzerine tek seferde bir kova büyüyebilir mi?

Kısa bir tarihçe — Fagin vd., 1979

- Ronald Fagin, Jürg Nievergelt, Nicholas Pippenger, H. R. Strong
- "Extendible Hashing — A Fast Access Method for Dynamic Files"
- Ana fikir: dizin (bellek) veri kovalarından (disk) ayrı
- Büyüme, ucuz dizine, disk sayfalarından çok daha sık dokunur

Fikir: yalnız gerektiğinde katlanan bir dizin

- Dizin: `2^global_depth` işaretçi, son bitlerle indekslenir
- Birden fazla girdi AYNI kovaya işaret edebilir
- Her kova kendi `local_depth` 'ini takip eder
- Taşma, `local_depth == global_depth` : dizin ÖNCE katlanır

Böl, sonra yeniden dene

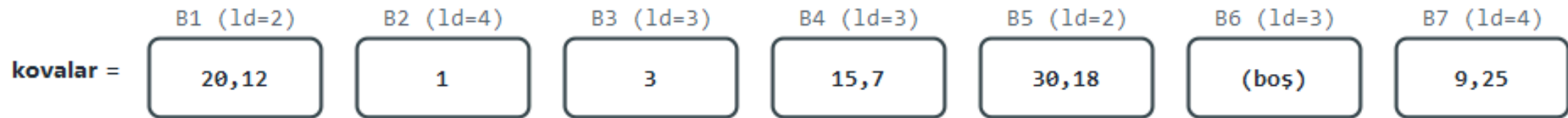
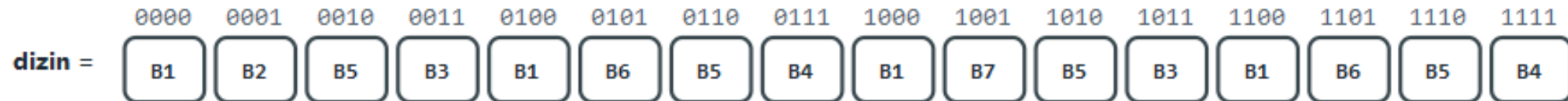
- Kova bölünür: `local_depth++` , yeni bir kova yaratılır
- Anahtarlar, daha derin bölünmenin incelediği TEK yeni bitle yeniden dağıtılır
- Eklemeyi yeniden dene: tek bir bölünme her zaman yetmez
- Düşmanca veri ART ARDA BİRKAÇ bölünme gerektirebilir

Genişleyebilir hashleme, adım adım

CAPACITY = 2

okuma: 16 yazma: 23

genel derinlik = 4



Bitti: 10 ekleme, global_depth=4 (dizin boyu 16), 7 kova, 16 okuma / 23 yazma. Dizin bellekte: her ekleme en çok 2 disk erişimi ister (oku + yaz, bölünmede +1 kova daha).

Uç durum — zincirleme bölünmeler



Bitti: 10 ekleme, global_depth=7 (dizin boyu 128), 10 kova, 19 okuma / 29 yazma. Dizin bellekte: her ekleme en çok 2 disk erişimi ister (oku + yaz, bölünmede +1 kova daha).

Kod — insert_key

```
void insert_key(Hash *h, int key) {
    int idx = last_bits(key, h->global_depth);
    Bucket *b = h->dir[idx];
    if (b->n < CAPACITY) { b->keys[b->n++] = key; return; }
    if (b->local_depth == h->global_depth) {
        h->global_depth++;
        double_directory(h);
    }
    split_bucket(h, b);
    insert_key(h, key);
}
```

Karmaşıklık

- $O(1)$ dizin araması (bellekte, ücretsiz)
- 1 sayfa okuması + 1 yazma, bölünme başına +2 daha
- Bölünmeler $O(1)$ amortize — dinamik dizi katlaması gibi
- Dizin yeri: $O(2^{\text{global_depth}})$, çarpık değilse küçük

Sık yapılan hatalar

- Dizinin katlanması gerekip gerekmediğini kontrol etmeden kovayı bölmek
- Yeniden denemeyi unutmak — tek bölünme anahtarları her zaman ayırmaz
- Bir bölünmenin yarattığı her kovanın boş olmayacağını varsaymak

Mini soru

Dizin neden bir kova bölünmeden ÖNCE katlanmalıdır, sonra değil?

Cevap

Bir bölünme, yönlendirmek için yedek,
daha özgül yuvalara ihtiyaç duyar. Önce
katlamadan, şu anki derinlikte hiçbiri yok.

9. Doğrusal Hashleme

Başlangıç sorusu

Genişleyebilir hashleme koca bir ek dizin yapısı ister. Bir dosya hiç dizin olmadan tek seferde bir kova büyüyebilir mi?

Kısa bir tarihçe — Witold Litwin, 1980

- "Linear Hashing: A New Tool for File and Table Addressing"
- Kovalar SABİT, sıralı (round-robin) düzende bölünür: 0, 1, 2...
- Gerçekte hangi kovanın taşıdığından tamamen bağımsız
- Bu öngörülebilirlik dizin ihtiyacını tamamen ortadan kaldırır

Fikir: bir sayaç n , sıradaki bölünmeyi takip eder

- `level` : kaç tam katlama turu tamamlandı
- `n` : bu turda kaç kova bölündü
- Adres: `key mod (N0 * 2level)`
- O adres `n` 'den küçükse (zaten bölünmüş): bir seviye derin kullan

HERHANGİ bir taşma, taşan değil, kova n'i böler

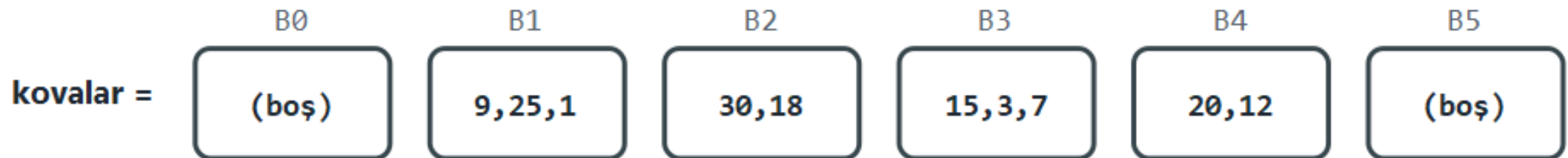
- Ekleme her zaman sığar: taşan bir kova basitçe büyür
- Herhangi bir yerde herhangi bir taşma, kova `n`'in bölünmesini tetikler
- `n` ilerler; tam bir tur `n=0`'a sıfırlar, `level++` yapar
- Bir kova geçici olarak CAPACITY'den fazla tutabilir

Doğrusal hashleme, adım adım

$N0 = 4$ CAPACITY = 2

okuma: 10 yazma: 18

seviye = 0 sıradaki bölünecek kova **$n = 2$**



Bitti: 10 ekleme, 6 kova (level=0, $n=2$), 10 okuma / 18 yazma. Dizin hiç gerekmedi -- yalnız basit bir mod işlemi ve bir sayaç (n).

Uç durum — sık bölünme, tam tur

$N0 = 2$ CAPACITY = 1

okuma: 10 yazma: 26

seviye = 2 sıradaki bölünecek kova n = 1



Bitti: 10 ekleme, 9 kova (level=2, n=1), 10 okuma / 26 yazma. Dizin hiç gerekmedi -- yalnız basit bir mod işlemi ve bir sayaç (n).

Kod — address ve split

```
int address(int key, int level, int n) {
    int a = key % (N0 << level);
    if (a < n) a = key % (N0 << (level + 1));
    return a;
}

void split(Hash *h) {
    int new_index = (N0 << h->level) + h->n;
    rehash_into(h, h->n, new_index);
    h->n++;
    if (h->n == (N0 << h->level)) { h->n = 0; h->level++; }
}
```

Karmaşıklık

- $O(1)$ ortalama ekleme — genişleyebilir hashlemeyle aynı
- SIFIR dizin belleği: yalnız iki tam sayı, level ve n
- Takas: tekil bir kovanın boyutu daha gevşek sınırlı
- Taşan ama sırası gelmemiş bir kova basitçe büyümeye devam eder

Sık yapılan hatalar

- Kova n yerine TAŞAN kovayı bölmek
- Bir adres hesaplarken $a < n$ düzeltmesini unutmak
- n 'i asla sıfıra sıfırlamamak ve level'i ilerletmemek

Mini soru

Doğrusal hashleme neden dizinden tamamen kaçınabilir, genişleyebilir hashleme kaçınamazken?

Cevap

Bölünme hedefi yalnız level ve n'den tamamen öngörülebilir. Hangi girdinin hangi kovaya işaret ettiğini kaydetmeye gerek yok.

10. Dış Birleştirmeli Sıralama

Başlangıç sorusu

Hafta 10 RAM'e sığan dizileri sıraladı.
Milyarlarca kayıtlık bir dosya sığmaz.
"Diğer yarı" diskte yaşadığında ne değişir?

Kısa bir tarihçe — bant çağı

- 1950–60'ların mainframe'leri RAM'den çok daha büyük dosyaları sıraladı
- Manyetik bant: yalnız sıralı erişim, rastgele atlama yok
- "Birleştirme" genelde mevcut olan TEK verimli işlemdi
- Knuth'un TAOCP Cilt 3'ü (1973): klasik, kapsamlı referans

Fikir: küçük sıralı çalışmalar, sonra k-yollu birleştirme

- Faz 1: RUN_SIZE parçaları, RAM'de sıralanır, ÇALIŞMA olarak yazılır
- Faz 2: FAN_IN çalışmayı birleştir, çalışma başına TEK arabellek
- Şu anki en küçük arabellek değerini seç, yaz, yeniden doldur
- Tam olarak TEK çalışma kalana dek geçişleri tekrarla

Yalnız kalan bir çalışma dokunulmadan taşınır

- Bir grupta yalnız 1 çalışma kaldıysa, birleşecek bir şey yok
- Bir sonraki geçişe dokunulmadan taşı — sıfır G/Ç
- Tek bir çalışmaya okuma+yazma harcamak sık görülen bir hatadır
- Bu iyileştirme tek sayıda çalışmada en çok önemlidir

Dış birleştirmeli sıralama, adım adım

RUN_SIZE = 4 FAN_IN = 2

okuma: 7 yazma: 5

sırasız girdi =

40, 11, 27, 8, 33, 2, 45, 16, 38, 23, 5, 30

geçiş 0 (çalışmalar) =

8, 11, 27, 40

2, 16, 33, 45

5, 23, 30, 38

geçiş 1

2, 8, 11, 16, 27, 33, 40, 45

5, 23, 30, 38

geçiş 2

2, 5, 8, 11, 16, 23, 27, 30, 33, 38, 40, 45

Bitti: 2 birleştirme geçişi, 7 okuma / 5 yazma. Sonuç tek bir sıralı çalışma: [2, 5, 8, 11, 16, 23, 27, 30, 33, 38, 40, 45]. Bir geçişte kaç çalışma taranırsa taransın, RAM her zaman yalnız FAN_IN arabellek kadar kullanılır.

Uç durum — RUN_SIZE=1, çok geçiş

RUN_SIZE = 1 FAN_IN = 2

okuma: 28 yazma: 19

sırasız girdi =

9, 3, 7, 1, 8, 2, 6, 4, 10, 5

geçiş 0 (çalışmalar) =

9

3

7

1

8

2

6

4

10

5

geçiş 1 =

3, 9

1, 7

2, 8

4, 6

5, 10

geçiş 2 =

1, 3, 7, 9

2, 4, 6, 8

5, 10

geçiş 3 =

1, 2, 3, 4, 6, 7, 8, 9

5, 10

geçiş 4 =

1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Bitti: 4 birleştirme geçişi, 28 okuma / 19 yazma. Sonuç tek bir sıralı çalışma: [1,2,3,4,5,6,7,8,9,10]. Bir geçişte kaç çalışma taranırsa taransın, RAM her zaman yalnız FAN_IN arabellek kadar kullanılır.

Kod — merge_group

```
Run merge_group(Run group[], int g) {  
    int ptr[FAN_IN] = {0};  
    Run out = new_run();  
    while (1) {  
        int best = -1, best_val = INT_MAX;  
        for (int i = 0; i < g; i++)  
            if (ptr[i] < group[i].n && group[i].keys[ptr[i]] < best_val)  
                { best_val = group[i].keys[ptr[i]]; best = i; }  
        if (best == -1) break;  
        append(&out, best_val);  
        ptr[best]++;  
    }  
    return out;  
}
```

Bu program GERÇEK dosyalar yaratır — güvenli

- Gerçek çalışma dosyaları, ama yalnız kendi kurduğu bir LAB KLASÖRÜNDE
- `lab_external_merge_sort/`, programın kendisi tarafından yaratılır
- Program çıkmadan önce her dosya, ve klasör, silinir
- Diskte başka hiçbir yere hiçbir şey yazılmaz

Karmaşıklık

- Faz 1: $\text{ceil}(n/\text{RUN_SIZE})$ çalışma, her biri 1 okuma + 1 yazma
- Geçişler: $O(\log_{\text{FAN_IN}}(n/\text{RUN_SIZE}))$
- Her geçiş her kayda bir kez dokunur: geçiş başına $O(n/B)$ G/Ç
- Toplam: $O((n/B) \cdot \log_{\text{FAN_IN}}(n/\text{RUN_SIZE}))$

Sık yapılan hatalar

- Yalnız kalan bir çalışmayı yine de birleştirmek (israf edilen okuma+yazma)
- Bir birleştirme sırasında tüm bir çalışmayı RAM'de tamponlamak
- RAM'in gerçekten tutabileceğinden büyük bir FAN_IN seçmek

Mini soru

Dış birleştirmeli sıralama neden dosya ne kadar dev olursa olsun yalnız $O(\text{FAN_IN})$ RAM arabelleğine ihtiyaç duyar?

Cevap

k-yollu bir birleştirme yalnız her çalışmanın **ŞU ANKİ** ön değerine ihtiyaç duyar. Çalışmalar zaten sıralıdır.

11. Yerine Koyarak Seçim: Daha Uzun Çalışmalar

Başlangıç sorusu

Bölüm 10'un Faz 1'i her zaman tam
RUN_SIZE uzunluğunda çalışmalar yapar.
Aynı RAM, RUN_SIZE'dan UZUN çalışmalar üretebilir mi?

Fikir: current ile next, son YAZMAYA göre kararlaştırılır

- Küçük bir RAM penceresi tut (üretimde bir min-heap)
- En küçük CURRENT-etiketli kaydı çıkar, yaz
- Yeni kayıt $\geq$ son yazılan ? CURRENT etiketle (çalışmayı uzatabilir)
- Yeni kayıt $<$ son yazılan ? NEXT etiketle (bir sonraki çalışmayı bekler)

En iyi durum, ortalama durum, en kötü durum

- Tüm pencere NEXT-etiketli: şu anki çalışma biter, yeniden etiketle, yeni çalışma
- Rastgele veri: çalışmalar ortalama pencere boyutu m'in yaklaşık **2 katı**
- Zaten sıralı girdi: dosya ne kadar büyük olursa olsun TEK çalışma
- Kesin azalan girdi: en kötü durum, çalışma başına tam olarak m

Yerine koyarak seçim, adım adım

RAM_SIZE = 4

okuma: 12 yazma: 12

girdi (sırayla okunacak) =

40, 11, 27, 8, 33, 2, 45, 16, 38, 23, 5, 30

RAM penceresi =

çıktı (geçerli çalışma) =

çalışma 1

8, 11, 27, 33, 40, 45

çalışma 2

2, 5, 16, 23, 30, 38

Bitti: 2 çalışma (uzunluklar: 6, 6), 12 okuma / 12 yazma. RAM_SIZE=4 olsa bile çalışmalar çoğu zaman 4'ten çok daha uzun -- rastgele veride ortalama ~2x.

Uç durum — en iyi durum, $\text{RAM} \geq n$

RAM_SIZE = 15

okuma: 10 yazma: 10

girdi (sırayla okunacak) =

9,3,7,1,8,2,6,4,10,5

RAM penceresi =

çalışma 1

çıktı (geçerli çalışma) =

1,2,3,4,5,6,7,8,9,10

Bitti: 1 çalışma (uzunluklar: 10), 10 okuma / 10 yazma. RAM_SIZE=15 olsa bile çalışmalar çoğu zaman 15'ten çok daha uzun -- rastgele veride ortalama $\sim 2x$.

Kod — extract_min_current

```
int extract_min_current(Item window[], int w) {  
    int best = -1;  
    for (int i = 0; i < w; i++)  
        if (window[i].tag == CURRENT &&  
            (best == -1 || window[i].val < window[best].val))  
            best = i;  
    return best;  
}
```

Bu program da GERÇEK dosyalar yaratır — güvenli

- `lab_replacement_selection/` , programın kendisi tarafından yaratılır
- Program çıkmadan önce her çalışma dosyası silinir
- Çalışma sınırları bu haftanın animasyonu ile tam eşleşir
- Üretim bir sürümü pencereyi gerçek bir min-heap olarak tutar

Karmaşıklık

- Çalışma yaratma için toplam $O(n)$ G/Ç — Bölüm 10 Faz 1'iyle aynı
- Rastgele veride $E[\text{çalışma uzunluğu}] \approx 2m$ — yarı kadar sonraki geçiş
- `extract_min_current`: burada $O(m)$, gerçek bir heap'le $O(\log m)$
- En kötü durum (azalan girdi): düz parçalamadan daha iyi değil

Sık yapılan hatalar

- Pencerenin minimumuna karşı karşılaştırmak, son YAZMAYA karşı değil
- Yeni bir çalışma başladığında last_written'ı sıfırlamayı unutmak
- "2x" ortalamasının en kötü durumda da geçerli olduğunu varsaymak

Mini soru

Yerine koyarak seçim neden ortalama olarak pencere boyutu m 'in yaklaşık İKİ KATI uzunlukta çalışmalar üretir?

Cevap

Herhangi bir anda pencerenin kabaca yarısı **CURRENT** kalır, sürekli tazelenir. Çalışma bitmeden önce ~2m'ye büyür.

12. Bir Dosya Organizasyonu Seçmek

Karşılaştırma tablosu (1/2)

Yapı	Arama	Ekleme
Sıralı dosya (H13)	$O(\log(n/B))$	$O(n/B)$ kaydırma
Kova hash (H13)	$\sim O(1)$ + taşma	$\sim O(1)$ + taşma
Birincil/ikincil dizin	$O(1)$ sayfa + ücretsiz tarama	Pahalı
ISAM	$O(L)$ + taşma zinciri	$O(L)$ + zincire ekleme

Karşılaştırma tablosu (2/2)

Yapı	Aralık sorgusu	Yeniden düzenleme?
B-ağacı	Tekrarlanan inişler	Asla
B+ -ağacı	Mükemmel (zincir)	Asla
Genişleyebilir hashleme	Zayıf (sıra yok)	Asla
Doğrusal hashleme	Zayıf (sıra yok)	Asla

Büyük tek karar

- SIRAYA mı ihtiyacınız var (aralık sorguları, sıralı gezinme)?
- Yoksa yalnız TAM-EŞLEŞME aramaları mı?
- Sıra önemliyse: bir B+-ağacının yaprak zincirini yenmek zor
- Yalnız tam-eşleşme, sürekli büyüyen dosya: hashleme genelde kazanır

Özet (1/2)

- **Dizinler:** birincil (seyrek), ikincil (yoğun), ISAM
- ISAM, dizin SEVİYELERİ ve bir TAŞMA ALANI ekler
- **B-ağaçları** (Bayer/McCreight 1972): bölme, arama, ödünç/birleştirme
- **B+ -ağaçları:** yaprak ZİNCİRİ aralık sorgularını hızlandırır

Özet (2/2)

- **Genişleyebilir hashleme** (Fagin vd. 1979): katlanan bir dizin
- **Doğrusal hashleme** (Litwin 1980): dizin yok, sıralı bölünmeler
- **Dış birleştirmeli sıralama**: küçük çalışmalar, k-yollu birleştirme
- **Yerine koyarak seçim**: aynı RAM'den ~2 kat daha uzun çalışmalar

Kendini Sınama Özeti

1. Birincil bir dizin neden taranması SIFIR disk G/Ç'sine mal olur?

Bellekte kalacak kadar küçüktür — kayıt başına değil, SAYFA başına tek girdi.

2. Yoğun bir dizin neden İKİNCİL anahtara göre sıralı olmalıdır?

Veri dosyasının kendi sırası eşleşen kayıtların nerede olduğu hakkında hiçbir şey söylemez — yalnız dizinin kendi sırası söyler.

3. ISAM'ın taşma alanı, bir dosyanın hangi işlemi hemen yapmaktan kaçınmasına izin verir?

Her eklemede tüm dosyayı yeniden bölmek ya da yeniden düzenlemek — bedeli, zincirler uzadıkça bozulan arama.

4. Bir B-ağacı bölünmesi neden her zaman tam olarak TEK anahtarı yukarı iter?

Ortanca, bölünmeden sonra HiÇBİR yarıya tam sığmayan tek anahtardır — bu yüzden o yükseltilir.

5. B-ağacı araması neden en çok (yükseklik + 1) okumaya mal olur?

Her yaprak tam olarak aynı derinliktedir — ekleme ağacı yalnız kökten büyütür, hiç bir yapraktan değil.

6. Bir B-ağacı neden birleştirmeye karşı ödünç almayı tercih eder?

Ödünç alma yalnız 3 sayfaya dokunur ve hemen çözülür;
birleştirme ağaçta daha yukarı yayılabilir.

7. Bir B+-ağacının aralık sorgularını hızlandıran TEK yapısal değişiklik nedir?

Her yaprak sağ komşusuna bir next işaretçisi tutar — köke tırmanmaya hiç gerek olmayan bir zincir.

8. Genişleyebilir hashlemenin dizini neden bazen katlanmak zorunda kalır?

Yalnız taşan kovanın `local_depth`'i `global_depth`'e yetiştiğinde
— diğer kovalar etkilenmez.

9. Doğrusal hashleme neden taşan kovadan BAŞKA bir kovayı bölebilir?

Önceden, gerçekte hangi kovanın taşıdığından bağımsız, sabit, sıralı bir bölünme sırasına bağlanır.

10. Yerine koyarak seçim neden genelde pencere boyutu m'i ikiye katlar?

Pencere sürekli yeniden dolar: herhangi bir anda kabaca yarısı
CURRENT kalır, çalışma $\sim 2m$ 'ye büyür.

Gelecek hafta

Hafta 15 — Final Proje Gösterimleri

Bu hafta yeni algoritma yok: takımlar projelerinin dosya-organizasyonu ya da depolama bileşenini sunuyor, Hafta 16'nın final sınav döneminden önce.

Kaynaklar (1/2)

- Ders izlencesi, Hafta 14: `CEN207-2026-2027-Guz-Izlence.tr.md`
- Bayer, McCreight (1972). "Organization and Maintenance of Large Ordered Indexes"
- Fagin, Nievergelt, Pippenger, Strong (1979). "Extendible Hashing"
- Litwin (1980). "Linear Hashing: A New Tool for File and Table Addressing"

Kaynaklar (2/2)

- Knuth. *TAOCP, Cilt 3: Sorting and Searching*, 2. baskı
- Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*. MIT Press
- Sedgewick, Wayne. *Algorithms*, 4. baskı. Addison-Wesley
- [williamfiset/Algorithms](#) · Programiz DSA